MINI PROJECT REPORT

ON

“Academic Chatbot: Agile Delivery and DevOps Automation Using Docker, CI/CD Pipelines and Monitoring”

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING

BY

Swaraj Kanse (T2, 2511131)

Adit Mokashi (T2, 2511136)

Aryan Patil (T2, 2511138)

SUBJECT

AGILE SOFTWARE DEVELOPMENT AND DEVOPS

SUPERVISOR

Prof. Sanober Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE
UNIVERSITY OF MUMBAI | A.Y. 2025-26

THADOMAL SHAHANI ENGINEERING COLLEGE

Advocate Nari Gursahani Marg, 37th Rd, Off Linking Road, TPS III, Bandra (West), Mumbai - 400050

Department of Artificial Intelligence & Data Science

University of Mumbai | Academic Year 2025-26

Certificate

This is to certify that the Mini Project entitled **"Academic Chatbot: Agile Delivery and DevOps Automation Using Docker, CI/CD Pipelines and Monitoring"** is a bonafide work of **Swaraj Kanse (T2, 2511131), Adit Mokashi (T2, 2511136) and Aryan Patil (T2, 2511138)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science"** for the subject **Agile Software Development and DevOps** during the academic year 2025-26.

_____	_____	_____
Prof. Sanober Shaikh		
Supervisor	Head of Department	Principal

Date:

Place: Mumbai

Abstract

This mini project applies Agile and DevOps practices to the Academic Chatbot, a Retrieval-Augmented Generation (RAG) study assistant that answers questions from a student's own PDFs with page-level citations, solves mathematics exactly with SymPy, and generates summaries, quizzes and flashcards. The focus of this report is not only the application but the engineering process used to build, test, ship and operate it.

The team followed Agile Scrum with two-week sprints tracked in Jira (project key ACAD), using four sprints, eight user stories and Smart Commits enforced by Git hooks. The application is packaged with a multi-stage Docker image that runs as a non-root user and defines a health check. Docker Compose starts the chatbot together with Prometheus, Grafana and cAdvisor on a private bridge network with persistent volumes.

A seven-stage Jenkins pipeline performs checkout, Flake8 linting, Pytest with coverage, Docker build and tagging, a container security audit, Compose deployment and smoke testing with a Nagios health plugin. GitHub Actions workflows repeat linting, testing, image building and pull-request verification. Prometheus metrics record requests, latency, strict-mode refusals, indexed documents and active sessions, and are visualised in Grafana. Together these form a complete plan-code-build-test-deploy-monitor cycle.

Keywords: *Agile Scrum, DevOps, CI/CD, Docker, Docker Compose, Jenkins, GitHub Actions, Prometheus, Grafana, Nagios, Jira*

TABLE OF CONTENTS

Content		Page No.
Abstract		**ii**
List of Figures		**iii**
1.	**Introduction**	**1**
1.1	Background	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Team Roles and Responsibilities	2
2.	**Implementation**	**3**
2.1	Technologies Used	3
2.2	Code Structure	3
2.3	Agile Scrum Methodology	4
2.4	Docker Containerization	4
2.5	Docker Compose Architecture	5
2.6	CI/CD Pipeline	5
2.7	Monitoring and Observability	6
2.8	Testing Strategy	6
3.	**Results and Discussion**	**7**
3.1	Result	7
3.2	Discussion	10
4.	**Conclusion**	**11**
4.1	Conclusion	11
5.	**References**	**12**

LIST OF FIGURES

Figure		Page No.
Figure 3.1	Uploading a PDF and generating a summary	8
Figure 3.2	Page-cited answer with expanded Sources	8
Figure 3.3	Strict Grounded Mode refusal for an out-of-document question	9
Figure 3.4	Exact SymPy answer in Standard Mode	9
Figure 3.5	Generated flashcards	10

Chapter 1
Introduction

1.1 Background

Modern software is expected to change quickly without breaking. Agile methods such as Scrum deliver software in short iterations and respond to feedback, while DevOps joins development and operations through automation: every change is built, tested, packaged, deployed and monitored in a repeatable way. Continuous Integration (CI) merges and verifies code often, Continuous Delivery/Deployment (CD) releases verified builds automatically, and containers such as Docker make an application behave the same on every machine.

This mini project applies these ideas to the Academic Chatbot, a study assistant that uses Retrieval-Augmented Generation (RAG). Students upload PDFs; the system retrieves relevant passages with TF-IDF and token-overlap scoring, a Groq-hosted language model answers with file and page citations, SymPy solves mathematics exactly, and quizzes, summaries and flashcards are generated from the uploaded material. The application is written in Python with a Streamlit interface, and it is the system that the DevOps toolchain in this report builds, ships and observes.

1.2 Problem Statement

Without a defined process, a small student project suffers from familiar delivery problems:

- **“Works on my machine.”** Code works on one laptop but fails on another because of different Python versions and dependencies.
- **Manual delivery.** Builds, tests and deployments are run by hand, so steps are skipped and errors reach users.
- **No traceability.** Commits are not linked to requirements, so progress and responsibility are unclear.
- **No visibility.** Without metrics and health checks, slow responses, failures and unusual behaviour are noticed late.

The problem is therefore to deliver the Academic Chatbot through an Agile process and an automated DevOps pipeline that packages it consistently, verifies every change, deploys the full stack with one command and monitors its health and behaviour, including how often strict grounded mode refuses an out-of-document question.

1.3 Objectives

- **Agile:** Plan and track the work as Agile Scrum sprints with user stories in Jira (project key ACAD).
- **Containerization:** Package the application as a secure multi-stage Docker image that runs as a non-root user and has a health check.

- **Orchestration:** Use Docker Compose to deploy the chatbot with Prometheus, Grafana and cAdvisor on one network.
- **CI/CD:** Automate linting, testing, image building, security audit, deployment and smoke testing with Jenkins and GitHub Actions.
- **Monitoring:** Expose Prometheus metrics for requests, latency, refusals, documents and sessions, and add Grafana dashboards and a Nagios health check.
- **Source control:** Enforce Jira-linked commit messages through Git hooks and Smart Commits.

1.4 Team Roles and Responsibilities

Member	Role	Responsibilities
Swaraj Kanse	Product Owner, Lead AI Architect	Core RAG architecture, grounded retrieval, AI tool integration, GitHub Actions CI/CD, source-control management, Jira workflow integration
Adit Mokashi	Scrum Master, DevOps Lead	Multi-stage Docker containerization, Docker Compose architecture, Jenkins pipeline, deployment automation, infrastructure configuration
Aryan Patil	SRE and Quality Assurance Lead	Prometheus instrumentation, Grafana dashboards, Nagios plugins, operational health checks, monitoring-related testing

Chapter 2
Implementation

2.1 Technologies Used:

Area	Tools	Purpose
Application	Python, Streamlit, Groq API, pypdf, scikit-learn, SymPy	RAG chatbot, exact maths, study tools
Containers	Docker (multi-stage build), Docker Compose	Consistent packaging and multi-service deployment
CI/CD	Jenkins (Jenkinsfile), GitHub Actions (ci-cd.yml, pr-verify.yml)	Automated lint, test, build, deploy and verify
Monitoring	prometheus-client, Prometheus, Grafana, cAdvisor, Nagios plugin	Metrics, dashboards, container telemetry and health checks
Testing and quality	Pytest, pytest-cov, JUnit XML, Flake8	Unit tests, coverage and static analysis
Agile and source control	Jira Software, Jira Smart Commits, Git hooks, GitHub	Sprint planning, traceable commits, repository hosting

2.2 Code Structure

```
Academic-Chatbot/
├── app.py                Streamlit UI, chat flow, metrics integration
├── rag_engine.py         RAG pipeline, tool calling, SymPy, study tools
├── entrypoint.py         Starts metrics server, launches Streamlit
├── requirements.txt      Python dependencies
├── .env.example          Groq / YouTube API key template
├── Dockerfile            Multi-stage production image
├── docker-compose.yml    App, Prometheus, Grafana, cAdvisor
├── docker-compose.jenkins.yml   Containerized Jenkins
├── Jenkinsfile           Seven-stage CI/CD pipeline
├── .github/workflows/    ci-cd.yml, pr-verify.yml
├── .githooks/            pre-commit, commit-msg
├── monitoring/
│   ├── metrics.py        Prometheus instrumentation
│   ├── prometheus/       prometheus.yml, alert.rules.yml
│   ├── grafana/          dashboards/, provisioning/
│   └── nagios/           check_chatbot_health.py, chatbot_service.cfg
├── tests/                test_rag_engine.py, test_monitoring.py, conftest.py
├── jira/                 sprint lifecycle JSON, jira_sync_helper.py,
│                         jira_web_server.py
├── docs/                 DEVOPS_ARCHITECTURE.md, MONITORING_GUIDE.md,
│                         AGILE_JIRA_SPRINT_LIFECYCLE.md
└── run.bat               Windows launcher
```

The application code (app.py, rag_engine.py) is kept separate from the delivery code (Dockerfile, Compose files, Jenkinsfile, workflows) and the operations code (monitoring/), so each concern can change independently. entrypoint.py is the container start-up script: it starts the Prometheus metrics server and then launches Streamlit.

2.3 Agile Scrum Methodology

The team used Scrum with two-week sprints. The Jira project (key ACAD) holds four sprints, eight user stories with story points, acceptance criteria, assignees and DevOps-related epics. Adit Mokashi acted as Scrum Master and Swaraj Kanse as Product Owner.

Sprint	Goal	Deliverables
Sprint 1 Grounded RAG Foundation	Build the retrieval engine, document provenance and strict grounding rules	Hybrid TF-IDF retrieval, PDF page tracking, out-of-domain refusal
Sprint 2 Academic UI and Formula Formatting	Improve the interface, add mathematical formatting and study tools	Streamlit UI improvements, LaTeX rendering, summary, flashcards and quiz generation
Sprint 3 Containerization and Automation	Dockerize the project and automate delivery	Multi-stage Dockerfile, Docker Compose architecture, Jenkinsfile, GitHub Actions workflows
Sprint 4 Observability and SRE	Add monitoring, dashboards and health checks	Prometheus instrumentation, Grafana dashboard, Nagios health plugin, alert rules

Jira Smart Commits and Git hooks. Every commit message must reference a Jira ticket. The pre-commit and commit-msg hooks in .githooks/ enforce this, and Smart Commit keywords update the ticket from Git, for example:

```
git commit -m "[ACAD-101] #comment Optimized multi-stage Docker build #time
2h #resolve"
```

The helper script jira/jira_sync_helper.py supports --board, --summary and --validate-commit options, so a commit message can be checked before it is pushed.

2.4 Docker Containerization

The Dockerfile uses a multi-stage build to keep the final image small and secure.

- **Builder stage.** Starts from python:3.11-slim, installs the Python dependencies and stores the built wheels under /build/wheels. Build tools stay out of the final image.
- **Runtime stage.** Starts again from a fresh python:3.11-slim, installs only runtime utilities such as curl, installs the pre-built wheels and copies the application files.
- **Security.** The container runs as the unprivileged user appuser (UID 1000) instead of root.
- **Ports and health.** Ports 8501 (Streamlit) and 8000 (Prometheus metrics) are exposed, and a HEALTHCHECK calls http://localhost:8501/_stcore/health so Docker can report the container as healthy or unhealthy.
- **Start-up.** The container starts through entrypoint.py.

2.5 Docker Compose Architecture

docker-compose.yml starts four services on the custom bridge network academic-chatbot-network. Prometheus and Grafana keep their data in named Docker volumes, so it survives container restarts.

Service	Port	Responsibility
academic-chatbot-app	8501, 8000	Streamlit web application (8501) and Prometheus metrics endpoint (8000)
Prometheus	9090	Scrapes metrics from the chatbot, stores time series, applies alerting rules, retains data for 15 days
Grafana	3000	Dashboards using Prometheus as a data source: latency, throughput, service health, indexed chunks, grounded refusals
cAdvisor	8080	Container resource usage and infrastructure-level telemetry

A second file, docker-compose.jenkins.yml, runs Jenkins itself in a container at http://localhost:8082, so the CI server can be recreated on any machine with a single command.

2.6 CI/CD Pipeline

Jenkins. The Jenkinsfile defines a seven-stage pipeline:

1 SCM Checkout	2 Lint (Flake8)	3 Tests (Pytest)	4 Docker Build	5 Security Audit	6 Compose Deploy	7 Smoke Test

#	Stage	What it does
1	SCM checkout and environment sanity	Checks out the code and prints the branch, commit SHA and build ID
2	Static code analysis and linting	Installs Flake8 and checks the application and test directories for syntax errors and serious Python problems
3	Automated tests	Installs dependencies, runs Pytest, writes JUnit XML results and runs coverage on the monitoring package
4	Docker build and tagging	Builds the multi-stage image and tags it with the build number and latest, with build metadata
5	Container security audit	Starts a temporary container, checks its user ID and fails the build if it runs as root
6	Compose deployment	Stops older containers and starts the chatbot, Prometheus and Grafana services
7	Smoke testing and Nagios verification	Waits for start-up, checks the Streamlit health and metrics endpoints and runs the Nagios health plugin

GitHub Actions. Two workflows in .github/workflows/ automate the same quality gates on GitHub: ci-cd.yml performs linting, Pytest execution, Docker image building with Buildx caching, Compose-based smoke tests and release-oriented automation, and pr-verify.yml verifies every pull request.

2.7 Monitoring and Observability

monitoring/metrics.py instruments the application with prometheus-client and serves the metrics on port 8000.

Metric	Type	Meaning
chatbot_requests_total	Counter	Total requests, labelled by status and mode (grounded, standard, fallback, general)
chatbot_query_duration_seconds	Histogram	Query latency, with buckets from about 0.1 s to 30 s
chatbot_rag_refusals_total	Counter	Questions refused because the answer was not in the uploaded documents (a hallucination-prevention signal)
chatbot_documents_indexed_total	Gauge	Number of indexed documents
chatbot_document_chunks_indexed_total	Gauge	Number of text chunks held in memory
chatbot_active_sessions	Gauge	Active user sessions

Grafana dashboards show service status, total queries, strict-RAG refusals, indexed chunks, P50 and P95 latency, query throughput, and grounded versus standard traffic. Alerting rules are kept in alert.rules.yml and container resource usage comes from cAdvisor.

Nagios health check. monitoring/nagios/check_chatbot_health.py actively checks the Streamlit application and the metrics endpoint, measures response latency and returns the standard Nagios exit codes: 0 OK, 1 WARNING, 2 CRITICAL, 3 UNKNOWN. Jenkins runs it as part of stage 7.

```
python monitoring/nagios/check_chatbot_health.py -H localhost -p 8501 -m 8000
--check-metrics
```

2.8 Testing Strategy

- **RAG engine tests (test_rag_engine.py)** (run automatically in Jenkins and GitHub Actions) check the DocumentChunk structure, preservation of source and page metadata, chunking, TF-IDF retrieval, retrieval relevance and the strict-mode refusal when no documents are uploaded.
- **Monitoring tests (test_monitoring.py)** check that request counters increment, query-duration metrics are recorded, refusal counters increment, document and chunk gauges update, and that the Nagios plugin detects invalid or unreachable endpoints.

Chapter 3
Results and Discussion

3.1 Result

The DevOps stack described in Chapter 2 is started and verified with the commands below, which also show what each step is expected to produce.

#	Command or action	Expected outcome
1	export GROQ_API_KEY="..." then docker compose up -d	Chatbot (8501), Prometheus (9090), Grafana (3000) and cAdvisor (8080) containers start
2	pytest tests/ -v	RAG engine and monitoring tests pass
3	docker compose -f docker-compose.jenkins.yml up -d, then open http://localhost:8082	Jenkins starts and can run the seven-stage pipeline
4	Push a commit and open a pull request	GitHub Actions ci-cd.yml and pr-verify.yml run lint, tests and image build
5	python monitoring/nagios/check_chatbot_health.py -H localhost -p 8501 -m 8000 --check-metrics	OK status (exit code 0) with measured latency
6	python jira/jira_sync_helper.py --validate-commit "[ACAD-101] Implement Docker security"	Commit message accepted because it references a Jira ticket

The application itself was run locally (Streamlit at localhost:8501) and exercised with a lecture PDF using the sequence below. Each screenshot shows the actual behaviour of the chatbot; no accuracy percentage or latency benchmark is reported, because a formal evaluation has not been carried out.

Step	Action	Observed behaviour
1	Upload a lecture PDF and click Summary	Summary with concepts, definitions and formulas, with page references
2	Ask a question present in the PDF and expand Sources	Answer with file and page citations
3	Ask a question that is not in the PDF in Strict Grounded Mode	Refusal: the provided document(s) do not contain the information
4	Turn Strict Grounded Mode off and ask "Integrate x * exp(x)"	Exact result, e^x (x − 1) + C, shown in LaTeX
5	Generate flashcards	Flashcards with a core concept, key insight and page reference

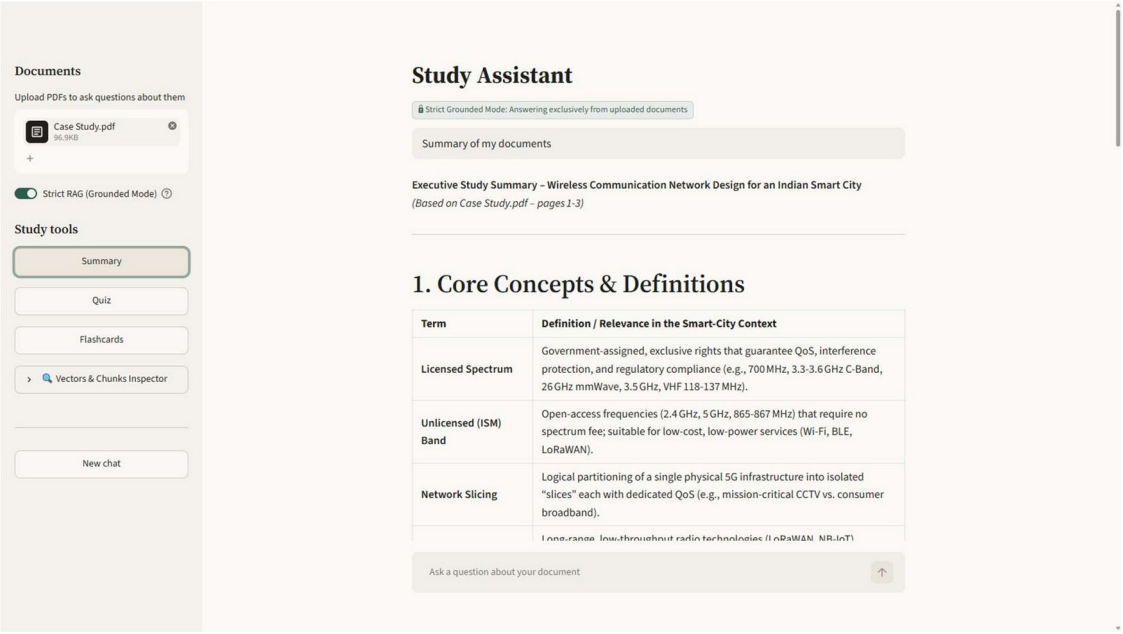


Figure 3.1: Uploading a PDF and generating a summary

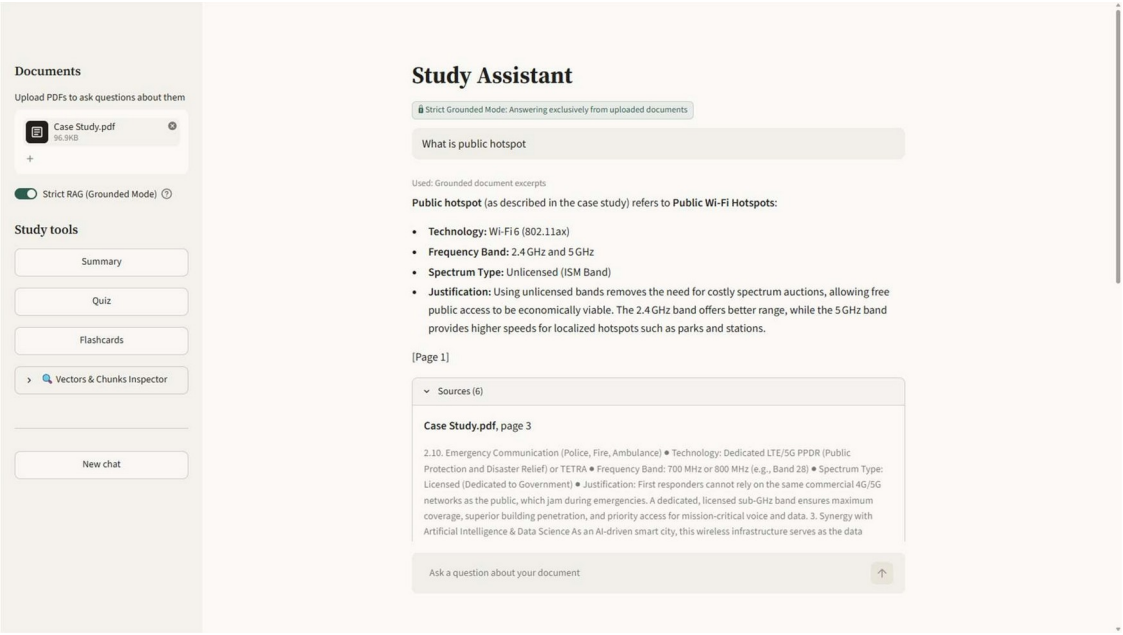


Figure 3.2: Page-cited answer with expanded Sources

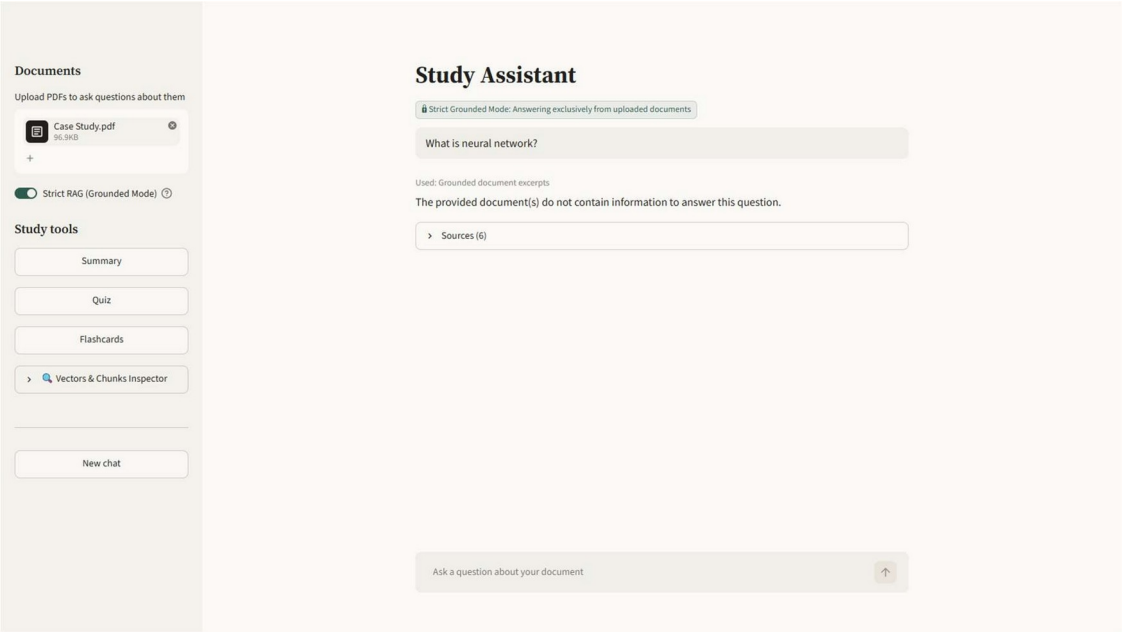


Figure 3.3: Strict Grounded Mode refusal for an out-of-document question

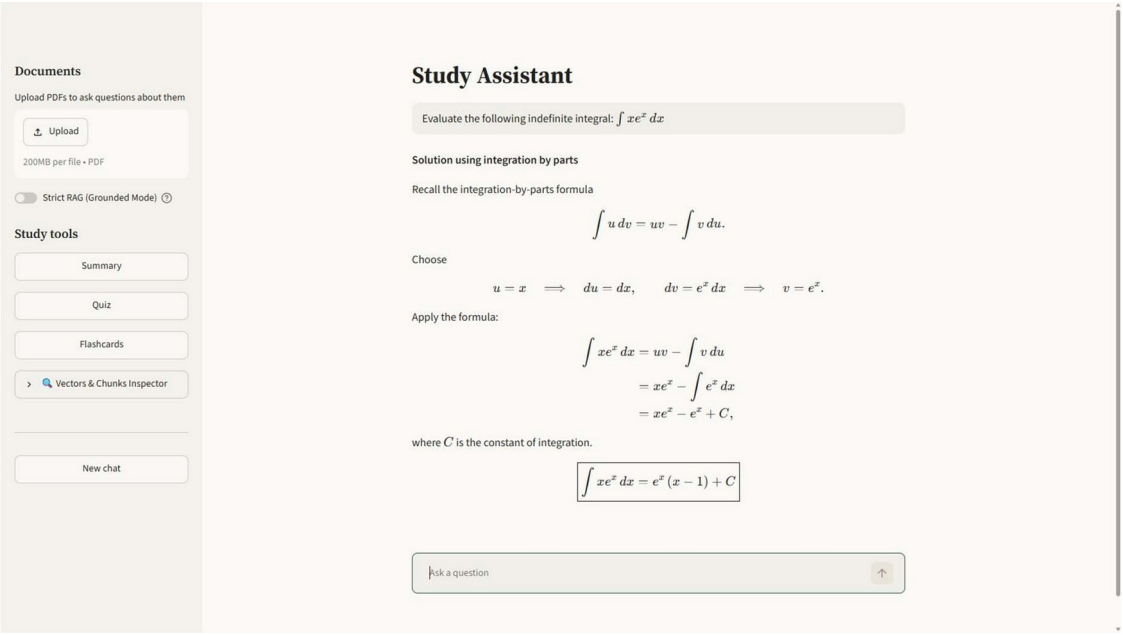


Figure 3.4: Exact SymPy answer in Standard Mode

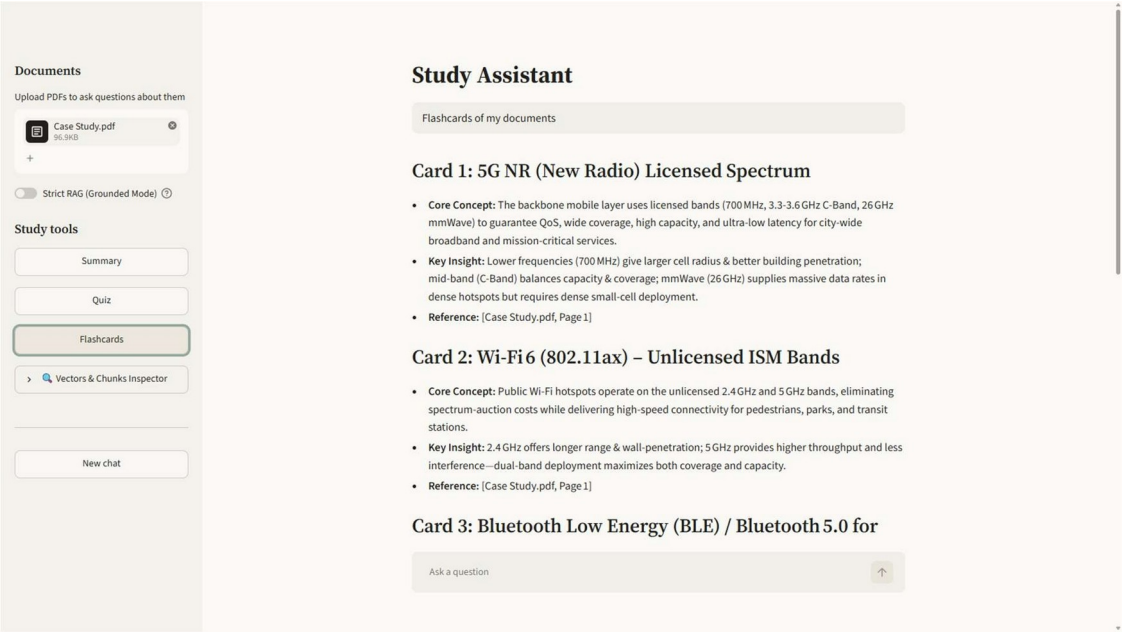


Figure 3.5: Generated flashcards

3.2 Discussion

The project shows the main DevOps ideas working together on one application. Agile sprints gave the work a clear order, from the RAG core in Sprint 1 to monitoring in Sprint 4, and Jira-linked commits make each change traceable to a story.

- **Consistency and security.** The multi-stage image gives the same runtime on every machine, and the non-root user and health check improve security and reliability. Stage 5 of the Jenkins pipeline fails the build if the container ever runs as root.
- **Automation.** Lint, tests, build, deployment and smoke tests run without manual steps, in Jenkins and in GitHub Actions, so broken code is caught before it is deployed.
- **Repeatable deployment.** The whole stack starts with one docker compose up -d, and Jenkins itself can be recreated from a Compose file.
- **Observability.** Metrics cover performance (latency, throughput), usage (documents, chunks, sessions) and AI safety (strict-mode refusals), and the Nagios plugin gives an active availability check.

The project also has limitations. Compose runs on a single host, so there is no orchestration, scaling or automatic failover. Grafana ships with the development login admin/admin, which must be changed outside a local demo, and API keys must be supplied through environment variables or a secret store, never committed. Uploaded documents are held in memory and are lost on restart. The tests give basic coverage of the RAG engine and the monitoring package only, there is no automated image vulnerability scan beyond the root-user audit, and the stack has not been deployed to a cloud environment.

Chapter 4
Conclusion

4.1 Conclusion

The Academic Chatbot was delivered as a complete Agile and DevOps project. Scrum sprints in Jira planned and tracked the work; Git hooks and Smart Commits kept every change linked to a ticket; a multi-stage, non-root Docker image packaged the application; Docker Compose deployed it with Prometheus, Grafana and cAdvisor; and Jenkins and GitHub Actions automated linting, testing, building, security checking, deployment and smoke testing. Prometheus metrics, Grafana dashboards and a Nagios health check let the team see how the system behaves, including how often strict grounded mode refuses an unsupported question.

Future work includes deploying the stack to Kubernetes or a cloud platform, automated image vulnerability scanning, secret management for credentials, wider automated test coverage, alert notifications to the team, and the application-level improvements already identified: semantic embeddings, OCR for scanned PDFs and persistent document storage.

Chapter 5
References

[1] K. Beck et al., “Manifesto for Agile Software Development,” 2001. https://agilemanifesto.org

[2] K. Schwaber and J. Sutherland, The Scrum Guide, 2020. https://scrumguides.org

[3] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley, 2010.

[4] G. Kim, J. Humble, P. Debois and J. Willis, The DevOps Handbook. IT Revolution Press, 2016.

[5] Docker Documentation (Dockerfile, multi-stage builds, Compose), https://docs.docker.com

[6] Jenkins Documentation (Pipeline and Jenkinsfile), https://www.jenkins.io/doc

[7] GitHub Actions Documentation, https://docs.github.com/actions

[8] Prometheus Documentation, https://prometheus.io/docs

[9] Grafana Documentation, https://grafana.com/docs

[10] Nagios Plugin Development Guidelines, https://nagios-plugins.org/doc/guidelines.html

[11] Atlassian Jira Software Documentation (Smart Commits), https://support.atlassian.com/jira-software-cloud

[12] pytest Documentation, https://docs.pytest.org; Flake8 Documentation, https://flake8.pycqa.org

[13] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, vol. 33, 2020.

[14] A. Meurer et al., “SymPy: symbolic computing in Python,” PeerJ Computer Science, vol. 3, e103, 2017.

[15] S. Kanse, A. Mokashi and A. Patil, Academic-Chatbot (source code), https://github.com/SwarajKanse/Academic-Chatbot